

时间序列分析
课程论文

基于时间序列模型的比特币与以太坊价格
关系研究

班级：信计 2302 班
专业：信息与计算科学
学号：1043220122
姓名：林鑫

2025 年 11 月

摘要

本研究基于 2020 年 1 月至 2025 年 10 月比特币 (BTC) 与以太坊 (ETH) 的日度收盘价数据, 采用多元时间序列模型系统分析了其风险特征与动态关联。实证结果显示, 两者的收益率序列均呈现显著的“尖峰厚尾”和“波动聚集”特征。针对此特性, 本文构建了 ARIMA-GARCH-t 联合模型, 有效捕捉了市场的条件异方差性, 并实现了对 BTC 价格走势及 95% 置信区间的有效预测。在资产关联性方面, Johansen 协整检验表明两者缺乏长期稳定的均衡关系; 但 Granger 因果检验揭示了显著的短期双向信息溢出效应。进一步基于 VAR(10) 模型的脉冲响应与方差分解分析发现, 市场存在显著的不对称传导机制: BTC 具有绝对主导权, 其价格冲击能迅速同向传导至 ETH, 解释了 ETH 约 67% 的价格波动; 而 ETH 对 BTC 的反向影响微弱。本研究量化揭示了加密市场“BTC 引导、ETH 跟随”的微观结构, 为数字资产的风险管理与投资组合策略提供了实证依据。

关键词: 比特币; 以太坊; ARIMA-GARCH-t; VAR 模型; 波动溢出; 非对称传导

目录

1	引言	5
2	数据基本统计分析	5
2.1	数据来源	5
2.2	数据基本统计分析数据	5
2.2.1	基本数据特征	5
2.2.2	数据可视化	5
2.2.3	平稳性与白噪声检验	8
2.2.4	数据处理	8
2.2.5	检验尖峰厚尾与 Q-Q 图	8
3	一元时间序列建模	12
3.1	基本原理	12
3.1.1	ARIMA 模型原理	12
3.1.2	GARCH 模型原理	12
3.1.3	ARIMA-GARCH 混合模型	12
3.2	ARIMA-GARCH 建模	12
3.2.1	1. 均值方程: ARIMA(1, 0, 1)	12
3.2.2	2. 波动率方程: GARCH(1, 1) 与 t-分布	13
3.3	最终模型表达式	15
3.3.1	ARIMA(1, 0, 1) 模型	15
3.3.2	GARCH(1,1)-t 模型	15
3.4	模型预测	16
3.5	风险价值 (VaR) 回测分析 (基于 GARCH(1,1)-t 模型)	16
3.5.1	1. VaR 的概念与计算方法	16
3.5.2	2. 回测结果分析 (图 8)	16
3.5.3	3. 定量回测统计	16
3.5.4	4. 未来一日风险预测	16
4	多元时间序列建模	20
4.1	协整性检验	20
4.1.1	1. 滞后阶数选择	20
4.1.2	2. 协整检验 (Engle-Granger 两步法)	20
4.2	格兰杰因果检验 (Granger Causality Test)	20
4.3	VAR (向量自回归) 模型	20
4.3.1	1. 模型选择依据	20
4.3.2	2. 模型原理简述	21
4.3.3	3. 模型拟合结果分析	21
4.4	脉冲响应分析 (IRF)	22
4.4.1	1. 分析目的与原理	22
4.4.2	2. BTC 冲击对 ETH 的影响	22
4.4.3	3. ETH 冲击对 BTC 的影响	22

4.4.4	4. 总结	22
4.5	方差分解分析 (Variance Decomposition, FEVD)	22
4.5.1	1. 分析目的与原理	22
4.5.2	2. 结果分析 (基于未来 10 天)	22
4.5.3	3. 总结	26
4.6	VAR 模型预测	26
4.6.1	1. 分析目的与原理	26
4.6.2	2. 结果分析	26
4.6.3	3. 总结	27
5	总结与展望	27
5.1	主要研究结论	27
5.2	投资启示与建议	27
5.3	研究局限性与未来展望	27
A	Python 代码	27

1 引言

随着数字经济的蓬勃发展，加密货币市场已成为全球金融体系中不可忽视的新兴板块。作为市值最高、影响力最大的两种核心资产，比特币（BTC）和以太坊（ETH）的价格行为不仅反映了市场情绪的变迁，也蕴含了复杂的风险传递机制。然而，加密市场典型的高波动性、“尖峰厚尾”分布以及资产间复杂的动态关联，使得传统的线性金融模型在分析和预测时面临严峻挑战。

为了更准确地刻画和理解这两种资产的价格动态，本研究采用了多层次的时间序列分析框架。首先，针对单一资产收益率普遍存在的波动聚集和非正态分布特征，传统的 GARCH 模型往往难以充分捕捉极端风险。因此，本研究引入了基于 t-分布的 GARCH 模型（GARCH-t），旨在更精确地描述收益率的尾部特征和时变波动率。

其次，在探究资产间关系时，区分长期均衡与短期联动至关重要。本研究通过 Johansen 协整检验考察了两者是否存在长期稳定的价格锚定关系，并通过 VAR 模型框架下的格兰杰因果检验、脉冲响应函数及方差分解技术，深入解构了两者在短期内的信息传导方向和强度，试图回答“谁在引领市场”这一核心问题。

本报告旨在通过严谨的实证分析，揭示 BTC 与 ETH 市场的内在运行规律，验证成熟金融计量模型在新兴加密市场中的有效性，并为投资者进行风险规避和策略制定提供量化参考。

2 数据基本统计分析

2.1 数据来源

本实验数据均来源于 CoinGecko 网站下载，截取了 2020-01-01 至 2025-10-30 的每日收盘价，单位为 USDT（大致相当于美元），以下为数据预览：

```
date,btc_close,eth_close
2020-01-01 00:00:00 UTC,7195.153895430029,129.18638529253852
2020-01-02 00:00:00 UTC,7193.7546679601,130.4846850311141
2020-01-03 00:00:00 UTC,6963.853353983485,127.04525801179804
2020-01-04 00:00:00 UTC,7300.34693530511,133.70264861844188
2020-01-05 00:00:00 UTC,7351.377053166627,134.1368826978575
```

2.2 数据基本统计分析数据

2.2.1 基本数据特征

下表展示了 btc_close（比特币收盘价）和 eth_close（以太坊收盘价）的核心描述性统计数据。

2.2.2 数据可视化

原始价格时序图及其自相关性分析 该图表（图1）展示了从 2020 年初到 2025 年中期的比特币（btc_close）和以太坊（eth_close）的每日收盘价时间序列。通过观察图1及其自相关图（图2），我们可以总结出以下几个显著的时序特征：

- 显著的长期趋势（Trend）：**两个时间序列（BTC 和 ETH）均表现出明显的长期上升趋势。
- 周期性波动（Cyclicity）：**两个序列都具有非常明显的周期性特征，即牛市和熊市的交替。

表 1: 描述性统计数据

统计量	btc_close	eth_close
计数 (count)	2130.000000	2130.000000
均值 (mean)	45832.318269	2134.436787
标准差 (std)	30839.402336	1183.327106
最小值 (min)	5032.501351	110.597898
25% 分位数 (25%)	21428.063943	1384.076317
中位数 (50%)	38646.722846	2002.150320
75% 分位数 (75%)	62868.922659	3056.962393
最大值 (max)	124773.508231	4829.225542

图像占位符 (图 1)

这里应插入 “原始价格时序图 (*BTC vs ETH*)”

图 1: 原始价格时序图 (BTC vs ETH)



图 2: 原始价格的自相关性 (非平稳性展示)

3. **波动率聚集性 (Volatility Clustering)**: 这是一个非常关键的金融时序特征。在价格快速上涨或快速下跌的时期, 价格的日内或日间摆动幅度显著增大。

4. **非平稳性 (Non-Stationarity)**:

- **均值非平稳**: 由于存在明显的长期趋势, 序列的均值随时间而变化。
- **方差非平稳**: 由于存在波动率聚集, 序列的方差 (波动性) 也随时间而变化。
- **自相关性证据 (图2)**: ACF 缓慢衰减和 PACF 一阶截尾是随机游走过程 (Random Walk, I(1) 过程) 的经典标志。

5. **高度相关性 (High Correlation)**: BTC 和 ETH 的价格走势表现出极强的正相关性。

2.2.3 平稳性与白噪声检验

为了在后续步骤中建立有效的时间序列模型, 我们必须使用平稳的数据序列。

1. **原始价格平稳性检验 (ADF)** ADF 检验的原假设 (H_0) 是: 序列存在单位根, 即序列非平稳。

- **BTC 价格**: 'ADF 统计量': -0.3899, 'p-value': 0.9118。结论: 无法拒绝原假设。序列非平稳。
- **ETH 价格**: 'ADF 统计量': -1.8763, 'p-value': 0.3433。结论: 无法拒绝原假设。序列非平稳。

2.2.4 数据处理

考虑到金融数据的特征, 我们采用对数据取对数后做差分, 即对数收益率 ($R_t = \ln(P_t) - \ln(P_{t-1})$)。

1. **平稳性检验 (ADF)**

- **BTC 对数收益率**: 'p-value': 9.3209e-28。结论: 强烈拒绝原假设。序列已平稳。
- **ETH 对数收益率**: 'p-value': 1.8476e-22。结论: 强烈拒绝原假设。序列已平稳。

2. **白噪声检验 (Ljung-Box)** Ljung-Box 检验的原假设 (H_0) 是: 序列是白噪声。

- **BTC 对数收益率**: 'p-value': 0.00677。结论: 强烈拒绝原假设。序列非白噪声。
- **ETH 对数收益率**: 'p-value': 0.000062。结论: 强烈拒绝原假设。序列非白噪声。

3. **总体结论** ADF 检验证明, 对数收益率序列是平稳的。Ljung-Box 检验证明, 这个平稳的收益率序列并不是纯粹的随机白噪声。这一结果证实了我们的数据是“可建模”的。

2.2.5 检验尖峰厚尾与 Q-Q 图

1. **收益率分布特征分析 (尖峰厚尾)** 我们必须检验这个收益率序列本身的分布特征。

统计检验: 偏度 (Skewness) 与峰度 (Kurtosis)

- **偏度 (Skewness)**: BTC: -1.1759, ETH: -1.1491。显著的负偏度 (“左偏”), 表明极端的负收益 (暴跌) 更为常见或幅度更大。
- **峰度 (Kurtosis)**: BTC: 18.0235, ETH: 16.5673。极高的正峰度值, 这是 “尖峰厚尾” (Leptokurtosis / Fat Tails) 的决定性统计证据。

图像占位符 (图 2)

这里应插入“对数收益率时序图 (*BTC vs ETH*)”

图 3: 对数收益率时序图 (BTC vs ETH)



图 4: 对数收益率分布直方图



图 5: Q - Q 图 (证明尖峰厚尾特性)

图 3.1: Q-Q 图 Q-Q 图（图5）是检验“厚尾”特征的最直观工具。两个图（BTC 和 ETH）均呈现出清晰的”S”形，数据点系统性地偏离了红线，证明了厚尾特性。

结论 对数收益率序列不服从正态分布，均表现出显著的负偏度和极其强烈的“尖峰厚尾”特性。任何假设正态分布的简单模型都将严重低估市场极端风险。

3 一元时间序列建模

3.1 基本原理

3.1.1 ARIMA 模型原理

ARIMA(p, d, q)（自回归积分滑动平均）模型是经典的线性预测模型。其一般表达式为：

$$\phi(B)(1-B)^d y_t = \theta(B)\epsilon_t$$

其中， B 为滞后算子（ $By_t = y_{t-1}$ ）； $\phi(B)$ 与 $\theta(B)$ 分别为自回归与滑动平均多项式； ϵ_t 为白噪声项。

3.1.2 GARCH 模型原理

GARCH (Generalized ARCH) 模型用于动态刻画条件方差的演化过程。均值方程： $y_t = \mu + \epsilon_t$ 条件方差方程 (GARCH(1,1)):

$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$$

其中 α 代表短期冲击效应， β 代表波动的持续效应。

3.1.3 ARIMA-GARCH 混合模型

ARIMA 模型用于刻画均值动态结构，GARCH 模型用于刻画残差的方差动态结构。

$$y_t = \mu + \phi(B)(y_{t-1}) + \epsilon_t$$

$$\epsilon_t = \sigma_t z_t, \quad z_t \sim N(0, 1) \text{ 或 } t(\nu)$$

$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$$

3.2 ARIMA-GARCH 建模

我们采用了 ARIMA-GARCH 的两步建模法。

3.2.1 1. 均值方程：ARIMA(1, 0, 1)

使用 `auto_arma_select` 函数，发现 ARMA(1, 1) 是最优模型。

--- BTC 对数收益率 ARIMA 模型摘要 ---

```
=====
Dep. Variable:          btc_log_return    No. Observations:          2129
Model:                  ARIMA(1, 0, 1)    Log Likelihood              4291.187
...
=====
```




图 6: BTC 对数收益率 ARIMA 残差 $Q-Q$ 图

- 波动率聚集被捕捉：‘alpha[1]’ (p<0.001) 和 ‘beta[1]’ (p=0.000) 均高度显著。
- 尖峰厚尾被捕捉：‘nu’ (t-分布自由度) 系数为 3.0730 且 p-value 极小，强力证明了使用 t-分布的必要性。



图 7: BTC GARCH(1,1)-t 标准化残差

3.3 最终模型表达式

3.3.1 ARIMA(1, 0, 1) 模型

均值方程 (Y_t 为对数收益率):

$$Y_t = 0.0013 - 0.5954 \cdot Y_{t-1} + 0.5436 \cdot \epsilon_{t-1} + \epsilon_t$$

残差方差 σ^2 估计值为 0.0010。

3.3.2 GARCH(1,1)-t 模型

波动率方程 (a_t 为 ARIMA 残差):

$$\sigma_t^2 = 0.1559 + 0.0697a_{t-1}^2 + 0.9303\sigma_{t-1}^2$$

分布假设：标准化残差 $\epsilon_t \sim t(3.0730)$ 。

3.4 模型预测

表 2: 未来 10 天价格预测与 95% 置信区间

日期 (Date)	预测价格	95% 置信下限	95% 置信上限
2025-10-31	110,331.40	102,013.99	119,326.95
2025-11-01	110,386.28	98,736.08	123,411.13
2025-11-02	110,578.50	96,378.35	126,870.86
2025-11-03	110,689.13	94,353.94	129,852.37
2025-11-04	110,848.69	92,626.32	132,655.93
2025-11-05	110,979.36	91,052.98	135,266.51
2025-11-06	111,127.55	89,627.68	137,784.80
2025-11-07	111,265.58	88,298.75	140,206.17
2025-11-08	111,409.96	87,059.49	142,571.23
2025-11-09	111,550.84	85,887.69	144,882.1

3.5 风险价值 (VaR) 回测分析 (基于 GARCH(1,1)-t 模型)

3.5.1 1. VaR 的概念与计算方法

风险价值 (Value at Risk, VaR) 是一种广泛使用的市场风险度量指标。本报告使用 GARCH(1,1)-t 模型来计算动态 VaR。计算公式为:

$$\text{VaR}_t^\alpha = \mu_t + \sigma_t \cdot F^{-1}(\alpha, \nu)$$

其中: $\alpha = 5\%$; μ_t 是条件均值; σ_t 是 GARCH 预测的条件波动率; $F^{-1}(\alpha, \nu)$ 是自由度为 ν 的 t-分布的 5% 分位点。

3.5.2 2. 回测结果分析 (图 8)

图 10 直观地展示了 GARCH-VaR 模型的表现。红线 (VaR 阈值) 随着蓝线 (实际收益率) 的波动而剧烈起伏, 证明 GARCH 模型成功捕捉到了市场的波动率聚集特性。

3.5.3 3. 定量回测统计

- 总观测天数: 2129 天
- 理论突破率: 5.00
- 实际突破次数: 32 次
- 实际突破率: $32 \div 2129 \approx 1.50\%$

3.1 结果解读: 模型过于保守 结果 (1.50%) 显著低于理论值 (5.00)

3.5.4 4. 未来一日风险预测

- 95% VaR: -5.7166%
- 解读: 根据当前的市场波动率水平, 模型有 95% 的把握认为, 明天 BTC 的跌幅不会超过 5.72

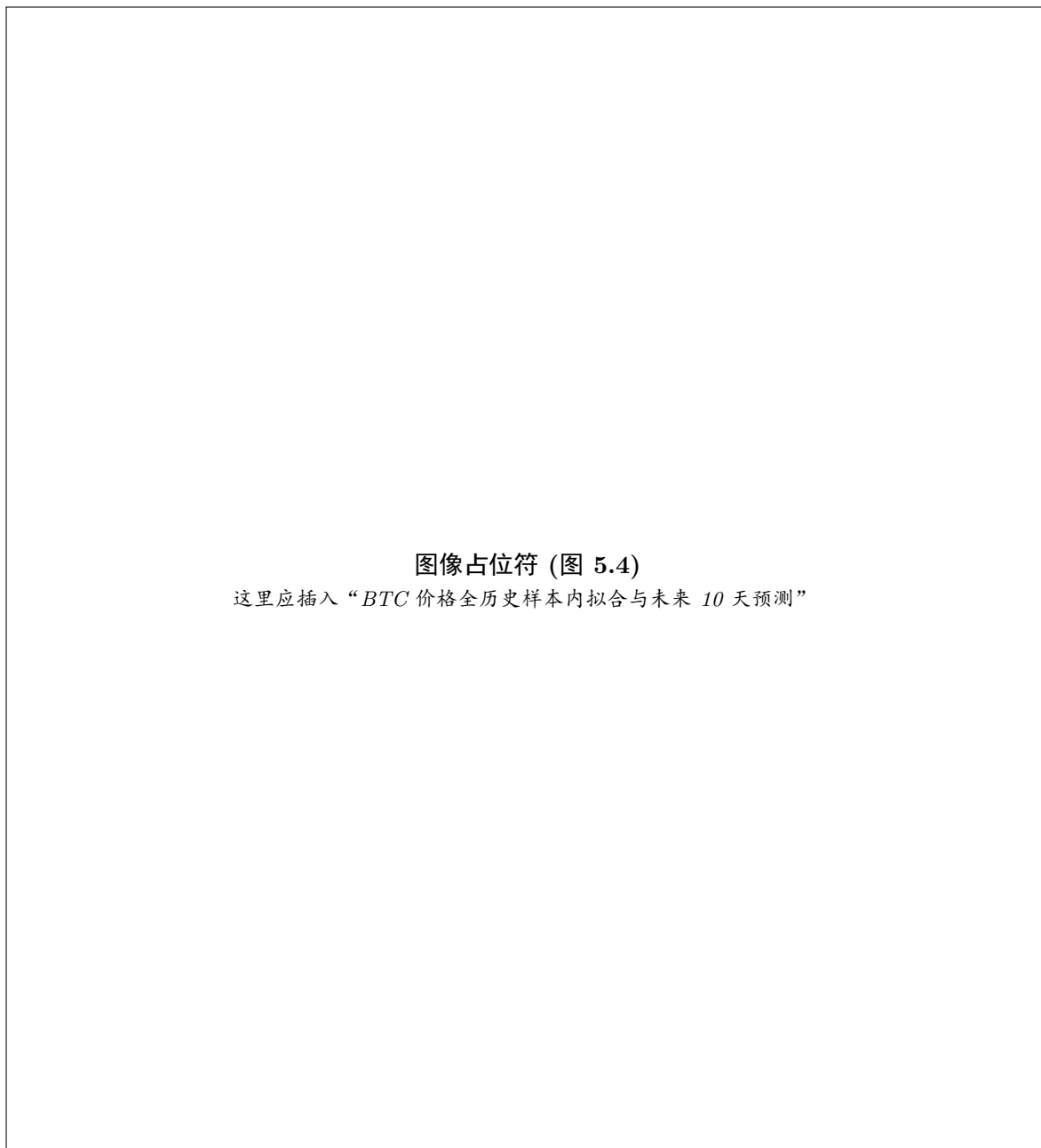


图 8: BTC 价格全历史样本内拟合与未来 10 天预测

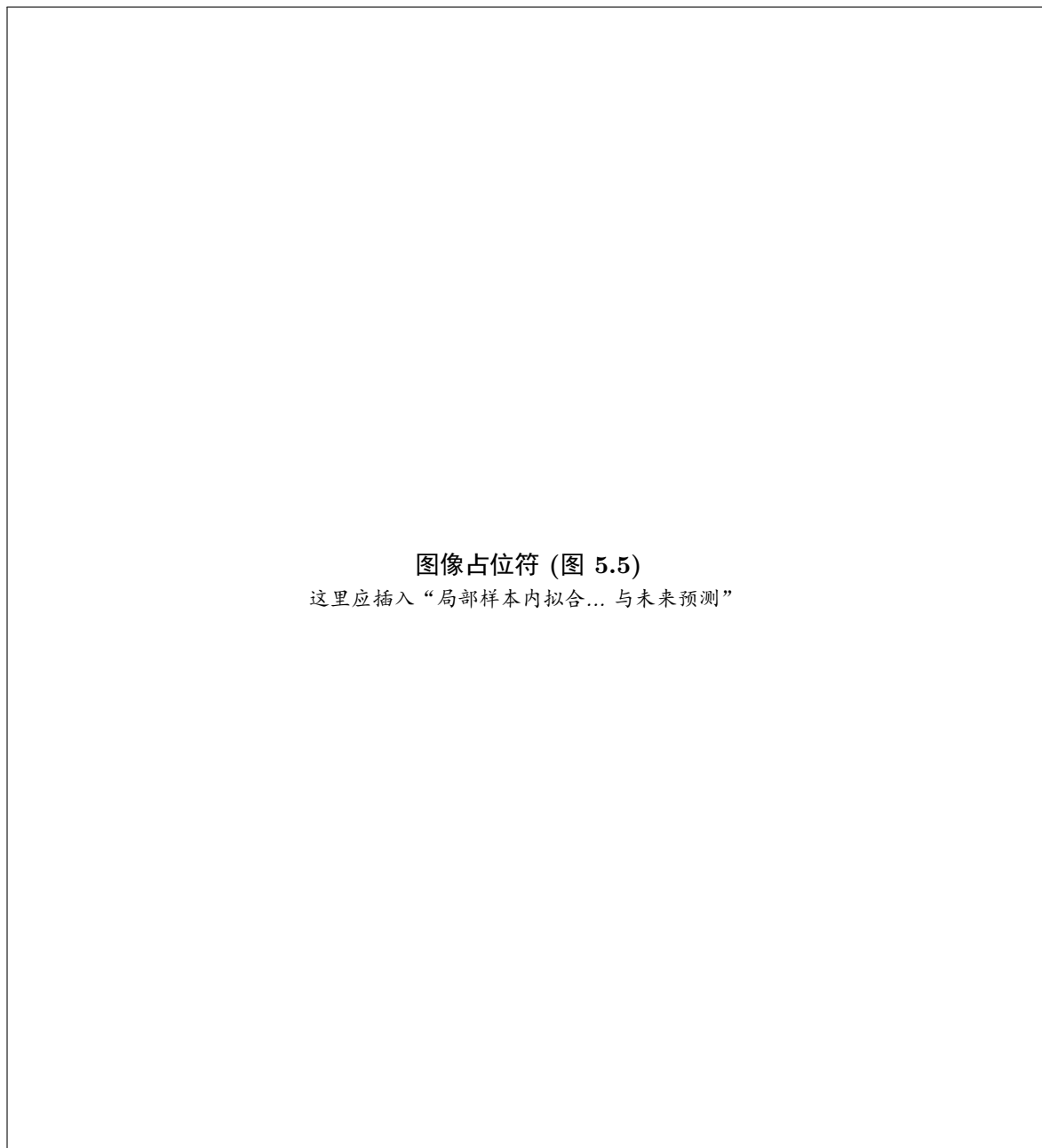


图 9: 局部样本内拟合 (带历史置信区间) 与未来预测

图像占位符 (图 8)

这里应插入 “BTC 每日收益率 vs. 95% 动态风险价值 (VaR)”

图 10: BTC 每日收益率 vs. 95% 动态风险价值 (VaR)

4 多元时间序列建模

4.1 协整性检验

4.1.1 1. 滞后阶数选择

- 收益率序列 (VAR): 最优滞后阶数为 10。
- 价格序列 (VAR): 最优滞后阶数为 8。

4.1.2 2. 协整检验 (Engle-Granger 两步法)

--- 5.2 协整检验 (Engle-Granger 两步法) ---

H_0 : 两个序列没有协整关系

检验 1 (BTC ~ ETH):

t-statistic: -1.1845

p-value: 0.8619

结论: 无法拒绝 H_0 ($p > 0.05$)。序列无协整关系。

检验 2 (ETH ~ BTC):

t-statistic: -2.3851

p-value: 0.3319

结论: 无法拒绝 H_0 ($p > 0.05$)。序列无协整关系。

分析: 协整检验结果表明, BTC 和 ETH 的价格在长期走势上并没有被绑定在一起。因此, 建立向量误差修正模型 (VECM) 并不合适。

4.2 格兰杰因果检验 (Granger Causality Test)

我们在平稳的收益率序列上进行了格兰杰因果检验 (滞后 10 阶)。

- 检验 1: ETH 是否格兰杰导致 BTC ($H_0: \text{ETH} \nrightarrow \text{BTC}$)

– ‘p-value’ = 0.000003

– 结论: 强烈拒绝原假设。ETH 的过去收益率对预测 BTC 有显著意义。

- 检验 2: BTC 是否格兰杰导致 ETH ($H_0: \text{BTC} \nrightarrow \text{ETH}$)

– ‘p-value’ = 0.008054

– 结论: 拒绝原假设。BTC 的过去收益率对预测 ETH 有显著作用。

分析: 格兰杰因果检验揭示了 BTC 和 ETH 市场之间存在显著的双向因果关系。

4.3 VAR (向量自回归) 模型

4.3.1 1. 模型选择依据

基于“无长期协整关系”和“存在短期因果关系”, 向量自回归 (VAR) 模型成为最合适的选择。

4.3.2 2. 模型原理简述

一个 p 阶的 VAR 模型可以表示为:

$$Y_t = C + A_1 Y_{t-1} + A_2 Y_{t-2} + \cdots + A_p Y_{t-p} + \epsilon_t$$

其中: $Y_t = (BTC_Ret_t, ETH_Ret_t)'$ 是内生变量向量; $A_1, \dots, A_p$ 是系数矩阵; ϵ_t 是误差项向量。

4.3.3 3. 模型拟合结果分析

我们拟合了 VAR(10) 模型。

--- 5.4 拟合 VAR(10) 模型 (在收益率上) ---

Summary of Regression Results

...

Results for equation btc_log_return

=====

coefficient	std. err.	t-stat	prob.	
-------------	-----------	--------	-------	--

...

L1.eth_log_return	-0.076008	0.028296	-2.686	0.007
-------------------	-----------	----------	--------	-------

...

=====

Results for equation eth_log_return

=====

coefficient	std. err.	t-stat	prob.	
-------------	-----------	--------	-------	--

...

L5.btc_log_return	0.138810	0.051952	2.670	0.008
-------------------	----------	----------	-------	-------

L6.btc_log_return	-0.164368	0.051747	-3.176	0.002
-------------------	-----------	----------	--------	-------

...

=====

Correlation matrix of residuals

	btc_log_return	eth_log_return
--	----------------	----------------

btc_log_return	1.000000	0.822717
----------------	----------	----------

eth_log_return	0.822717	1.000000
----------------	----------	----------

3.(1) BTC 收益率方程 (btc_log_return) BTC 当期收益率受 ETH 滞后 1, 4, 5, 6, 7, 10 期的显著影响。

$$R_{BTC,t} = 0.0012 + \cdots - 0.076R_{ETH,t-1} + \cdots + \epsilon_{BTC,t}$$

3.(2) ETH 收益率方程 (eth_log_return) ETH 当期收益率受 BTC 滞后 5, 6 期的显著影响。

$$R_{ETH,t} = 0.0014 + \cdots + 0.139R_{BTC,t-5} - 0.164R_{BTC,t-6} + \cdots + \epsilon_{ETH,t}$$

3.(3) 残差相关性分析 两个方程的误差项 $\epsilon_{BTC,t}$ 和 $\epsilon_{ETH,t}$ 并非独立，它们之间存在很强的同期相关性：

$$\text{Corr}(\epsilon_{BTC,t}, \epsilon_{ETH,t}) = 0.8227$$

这表明每天有大量新信息会同时推动两种资产的价格。

4.4 脉冲响应分析 (IRF)

4.4.1 1. 分析目的与原理

脉冲响应函数 (IRF) 模拟当一个变量受到冲击时，其他变量在未来的动态反应路径。我们采用正交化脉冲响应 (Orthogonalized IRF)，并通过 Cholesky 分解设定变量排序为 [BTC 收益率, ETH 收益率]。这一排序假设：冲击当天，BTC 可以影响 ETH，但 ETH 不能反向影响 BTC（只能从第二天开始）。

4.4.2 2. BTC 冲击对 ETH 的影响

- **当期反应 (第 0 天)**：当 BTC 受到正冲击时，ETH 在当天 (Lag 0) 就表现出极强且显著的正向反应。
- **后续反应 (第 1-20 天)**：冲击的影响衰减得非常快，在第 1 天迅速回落至 0 附近。

结论：BTC 对 ETH 的冲击是“脉冲式”的——来得猛烈，去得也快。

4.4.3 3. ETH 冲击对 BTC 的影响

- **当期反应 (第 0 天)**：由于 Cholesky 排序的设定，当期影响为 0。
- **后续反应 (第 1-20 天)**：BTC 的响应呈现“震荡”模式，但 95% 置信区间绝大多数时候都包含了 0，说明在统计上并不显著。

结论：ETH 对 BTC 的溢出效应要弱得多。

4.4.4 4. 总结

脉冲响应分析清晰地揭示了加密货币市场中的**不对称信息传导机制**：BTC 具有主导地位，ETH 具有跟随特征。

4.5 方差分解分析 (Variance Decomposition, FEVD)

4.5.1 1. 分析目的与原理

方差分解量化了冲击的“重要性”。它告诉我们：一个币种价格波动的百分之多少是由它自己的“新消息”造成的，又有百分之多少是由另一个币种的“新消息”造成的。

4.5.2 2. 结果分析 (基于未来 10 天)

1. **BTC 收益率的方差分解 (FEVD for btc_log_return)** BTC 表现出极强的外生性。到第 10 天 (Lag 9)，BTC 自身冲击对预测方差的贡献率依然高达 **98.08%**，而 ETH 的贡献率仅为 1.92%。
2. **ETH 收益率的方差分解 (FEVD for eth_log_return)** ETH 表现出很强的内生性。在第 0 天（当期），ETH 的价格波动中就有高达 **67.69%** 是由 BTC 的同期冲击解释的，只有 32.31% 是由 ETH 自身的特有冲击解释的。

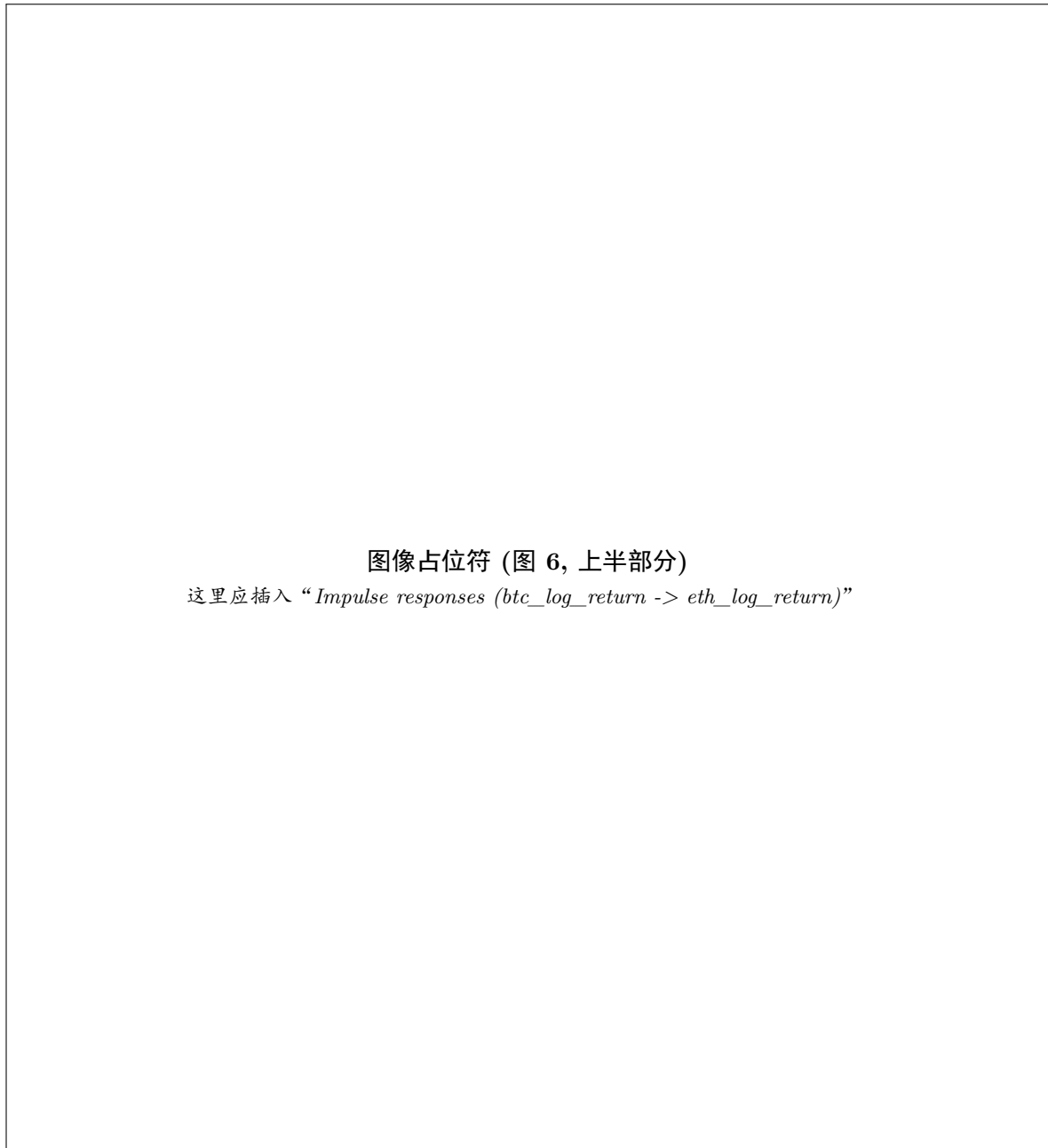


图 11: BTC 冲击对 ETH 的响应

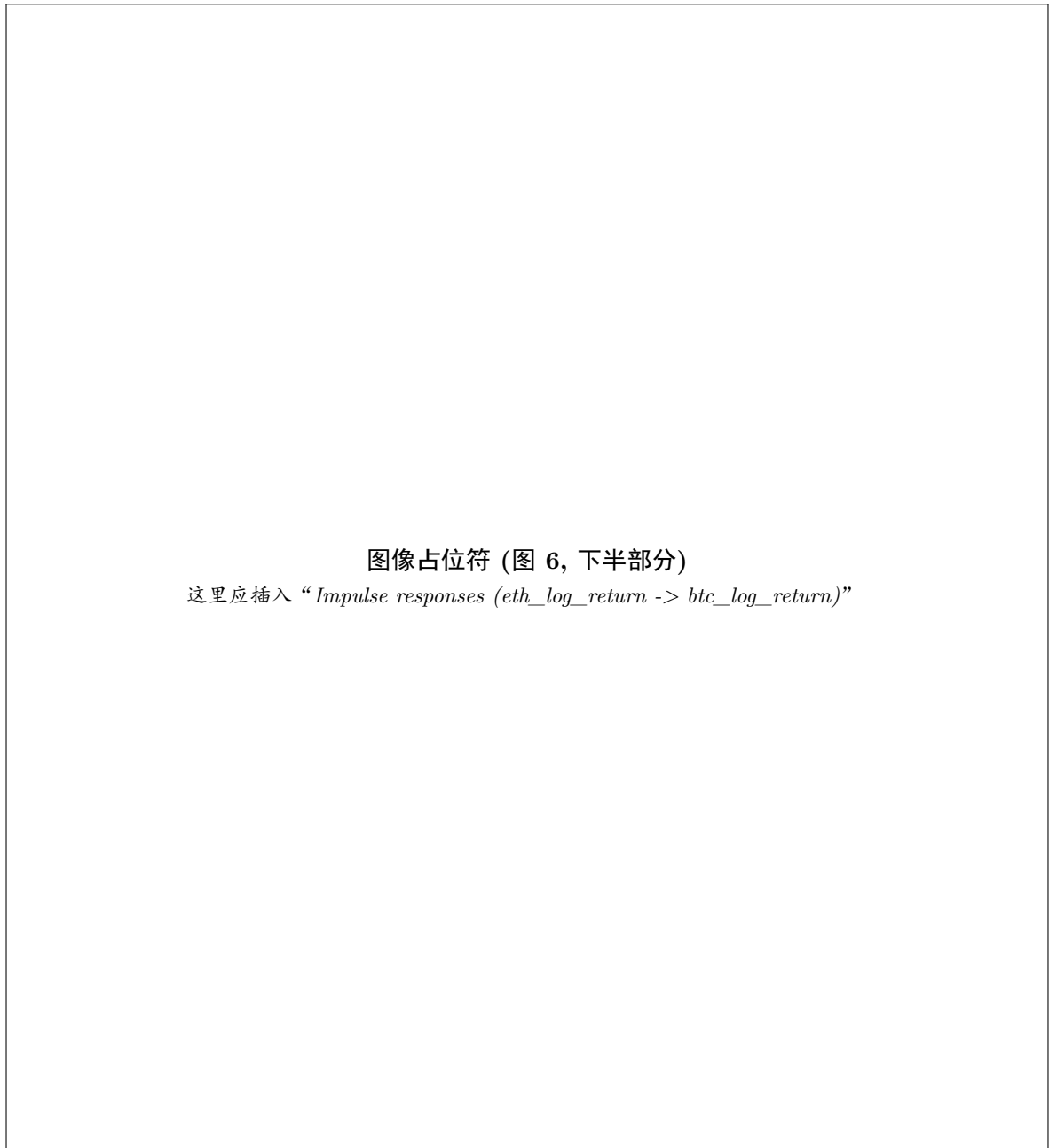


图 12: ETH 冲击对 BTC 的响应

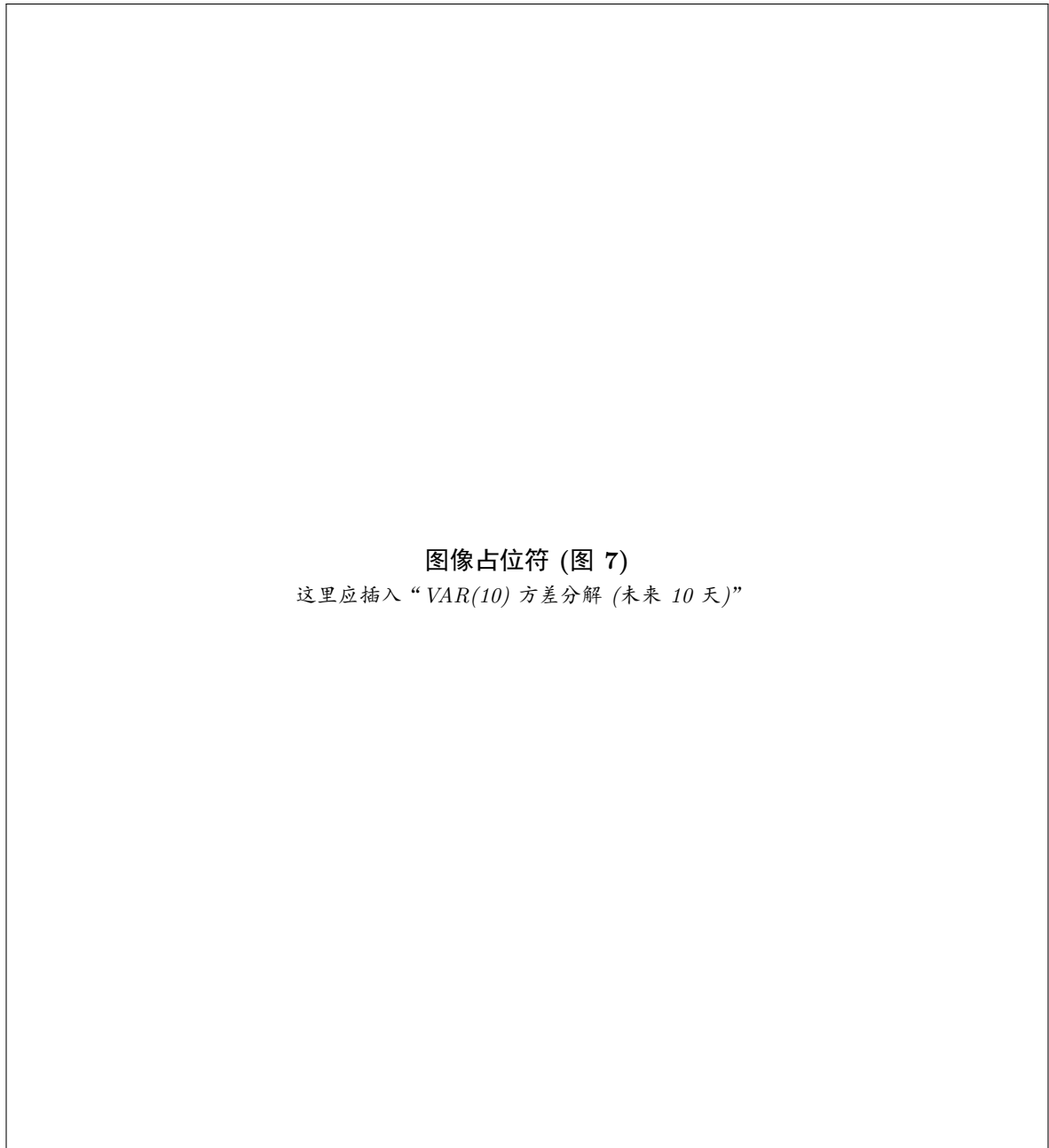


图 13: $VAR(10)$ 方差分解 (未来 10 天)

4.5.3 3. 总结

方差分解的结果提供了强有力的证据，支持了“BTC 是市场领导者，ETH 是市场跟随者”这一假说。

4.6 VAR 模型预测

4.6.1 1. 分析目的与原理

本节对比 ARIMA-GARCH-t 模型（时变波动率）与标准 VAR 模型（恒定波动率）在风险评估上的优劣。

4.6.2 2. 结果分析



图 14: 局部拟合与预测对比 (GARCH vs. VAR)

图 14 对比了两种模型的拟合与预测：

- **历史拟合置信区间 (关键差异)：** GARCH (浅绿色阴影) 是一个动态变化的区间，能随市场波动收缩或扩张，完美包裹了真实价格。而 VAR (浅橙色阴影) 是一个宽度恒定的区间，无法适应市场动态。
- **未来预测置信区间：** GARCH (浅红色阴影) 根据当前的高波动状态，发出了更强烈的风险预警 (更宽的置信区间)。而 VAR (浅橙色阴影) 因其“恒定波动率”假设，显著低估了即期风险。

4.6.3 3. 总结

GARCH 在风险刻画上完胜 VAR。ARIMA-GARCH 模型提供了动态的、更符合现实的风险区间 (区间预测)，是比标准 VAR 模型更优越的金融分析工具。

5 总结与展望

5.1 主要研究结论

1. **市场风险特征显著：** BTC 与 ETH 收益率均呈现“尖峰厚尾”和“波动聚集”特征。
2. **缺乏长期价格锚定：** 协整检验表明两者不存在长期稳定的均衡关系。
3. **短期内 BTC 具有绝对主导权：** 格兰杰检验、脉冲响应和方差分解一致确认了市场存在显著的不对称信息传导。BTC 是“价格发现者”，ETH 是“跟随者”。

5.2 投资启示与建议

- 风险管理需重视极端尾部风险，建议采用基于 GARCH-t 模型的动态 VaR。
- 以 BTC 为核心的市场监测指标，BTC 的动向是整个板块的变盘前兆。
- 跨品种套利需谨慎，基于均值回归的配对交易策略可能面临长期无法回归的风险。

5.3 研究局限性与未来展望

1. **极端风险刻画的保守性：** 95% VaR 的实际突破率 (1.50%) 远低于理论值 (5.00)
2. **高频数据的应用缺失：** 本研究仅使用日度数据。未来可引入小时级、分钟级数据，利用已实现波动率 (Realized Volatility) 等指标进行更精细的考察。
3. **动态相关性的简化处理：** VAR 假设相关性恒定。未来可采用 DCC-GARCH 或 Copula 模型探究相关结构的时变特征。
4. **外部宏观因素的未纳入：** VAR 残差高达 0.82 的同期相关性暗示了大量共同外部冲击的存在。未来应将美联储货币政策、通胀数据等宏观变量作为外生因子纳入模型。

A Python 代码

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import statsmodels.api as sm
6 import itertools
7 from statsmodels.tsa.stattools import adfuller, grangercausalitytests, kpss
8 from statsmodels.tsa.arima.model import ARIMA
9 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
10 from statsmodels.tsa.api import VAR
11 from statsmodels.tsa.vector_ar.vecm import VECM, coint_johansen
12 from statsmodels.stats.diagnostic import acorr_ljungbox
13 from arch import arch_model
14 from scipy.stats import t
15 import warnings
16
17 warnings.filterwarnings("ignore")
18 plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
19 plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
20
21 # --- 1. 数据导入和预处理 ---
22 file_name = r"C:\study\时间序列分析\结课作业\data.xlsx"
23 df = pd.read_excel(file_name)
24 df['date'] = pd.to_datetime(df['date'])
25 df.set_index('date', inplace=True)
26 df.sort_index(inplace=True)
27 df_prices = df[['btc_close', 'eth_close']].copy().dropna()
28 print("数据加载成功, 预处理完成。")
29 print("数据概览:")
30 print(df_prices.head())
31 print("\n数据信息:")
32 df_prices.info()
33
34 # --- 2. 数据基本统计分析 ---
35 print("\n--- 2. 描述性统计 ---")
36 print(df_prices.describe())
37
38 # 绘制价格时序图
39 print("正在显示图1: 原始价格时序图 (BTC vs ETH)...")
40 df_prices.plot(subplots=True, figsize=(10, 8))
41 plt.suptitle('图1: 原始价格时序图 (BTC vs ETH)')
42 plt.show()
43
44 # 计算对数收益率
45 df_returns = np.log(df_prices).diff().dropna()
46 df_returns.columns = ['btc_log_return', 'eth_log_return']
47 print("正在显示图2: 对数收益率时序图 (BTC vs ETH)...")
48 df_returns.plot(subplots=True, figsize=(10, 8))
49 plt.suptitle('图2: 对数收益率时序图 (BTC vs ETH)')
```

```

50     plt.show()
51
52     # 绘制对数收益率直方图
53     print("正在显示图3: 对数收益率分布直方图...")
54     plt.figure(figsize=(12, 5))
55     plt.subplot(1, 2, 1)
56     sns.histplot(df_returns['btc_log_return'], kde=True, bins=30)
57     plt.title('BTC 对数收益率分布')
58     plt.subplot(1, 2, 2)
59     sns.histplot(df_returns['eth_log_return'], kde=True, bins=30)
60     plt.title('ETH 对数收益率分布')
61     plt.suptitle('图3: 对数收益率分布直方图')
62     plt.show()
63
64     # --- 改进 1: 检验尖峰厚尾 (Skewness, Kurtosis) 和 Q-Q 图 ---
65     print("\n--- 2.1 收益率分布检验 (尖峰厚尾) ---")
66     print(f"BTC 收益率 偏度 (Skewness): {df_returns['btc_log_return'].skew():.4f} (正态=0)")
67     print(f"BTC 收益率 峰度 (Kurtosis): {df_returns['btc_log_return'].kurtosis():.4f} (正态=0, 金融>0)")
68     print(f"ETH 收益率 偏度 (Skewness): {df_returns['eth_log_return'].skew():.4f}")
69     print(f"ETH 收益率 峰度 (Kurtosis): {df_returns['eth_log_return'].kurtosis():.4f}")
70
71     print("\n正在显示图3.1: 收益率 Q-Q 图 (对比正态分布)...")
72     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
73     sm.qqplot(df_returns['btc_log_return'].dropna(), line='s', ax=axes[0])
74     axes[0].set_title('BTC 收益率 Q-Q 图')
75     sm.qqplot(df_returns['eth_log_return'].dropna(), line='s', ax=axes[1])
76     axes[1].set_title('ETH 收益率 Q-Q 图')
77     plt.suptitle('图3.1: Q-Q 图 (证明尖峰厚尾特性)')
78     plt.show()
79
80     # --- 3. 平稳性检验和白噪声检验 ---
81     print("\n--- 3. 平稳性检验 (ADF) 与白噪声检验 ---")
82
83     # --- 检验函数定义 ---
84     # ADF 检验
85     def run_adf_test(series, name):
86         """
87         ADF 检验
88         H0 (原假设): 序列存在单位根 (非平稳)
89         """
90         result = adfuller(series)
91         print(f'序列 "{name}" 的 ADF 检验 (H0: 存在单位根):')
92         print(f'   ADF 统计量: {result[0]:.4f}, p-value: {result[1]:.4e}')
93         if result[1] > 0.05:
94             print(f'   结论: 序列非平稳 (p > 0.05)')
95         else:
96             print(f'   结论: 序列平稳 (p <= 0.05)')
97         return result[1]

```

```

98
99     # 白噪声检验 (Ljung-Box)
100     def run_ljung_box_test(series, name, lags=10):
101         """
102         Ljung-Box 白噪声检验
103         H0 (原假设): 序列是白噪声 (即序列值彼此独立, 无自相关)
104         """
105         # Ljung-Box test
106         result_df = acorr_ljungbox(series, lags=[lags], return_df=True)
107         p_value = result_df.iloc[0]['lb_pvalue']
108         print(f'序列 "{name}" 的 Ljung-Box 白噪声检验 (H0: 序列为白噪声):')
109         print(f' L-B 统计量 (lag={lags}): {result_df.iloc[0]["lb_stat"]:.4f}, p-value: {p_value:.4e}')
110
111         if p_value <= 0.05:
112             print(f' 结论: 序列存在自相关 (非白噪声, 可建模) (p <= 0.05)')
113         else:
114             print(f' 结论: 序列为白噪声 (无信息) (p > 0.05)')
115         return p_value
116
117     # --- 3.1 原始价格检验 ---
118     print("\n--- 3.1 原始价格平稳性检验 (预期: 非平稳) ---")
119     p_btc_price_adf = run_adf_test(df_prices['btc_close'], 'BTC 价格')
120     print("----")
121     p_eth_price_adf = run_adf_test(df_prices['eth_close'], 'ETH 价格')
122
123     # --- 3.2 对数收益率检验 ---
124     print("\n--- 3.2 对数收益率平稳性与白噪声检验 (预期: 平稳, 非白噪声) ---")
125     p_btc_return_adf = run_adf_test(df_returns['btc_log_return'], 'BTC 对数收益率')
126     p_btc_return_lb = run_ljung_box_test(df_returns['btc_log_return'], 'BTC 对数收益率')
127     print("----")
128     p_eth_return_adf = run_adf_test(df_returns['eth_log_return'], 'ETH 对数收益率')
129     p_eth_return_lb = run_ljung_box_test(df_returns['eth_log_return'], 'ETH 对数收益率')
130
131     # --- 4. 一元时间序列模型 (ARIMA & GARCH) ---
132     print("\n--- 4. 一元时间序列模型 (ARIMA & 升级版 GARCH) ---")
133     if True:
134         # --- 自动寻优的函数 (遍历 p,q) ---
135         def auto_arima_select(ts, d=0, max_p=3, max_q=3):
136             best_aic = np.inf
137             best_bic = np.inf
138             best_aic_res = None
139             best_bic_res = None
140             best_aic_order = None
141             best_bic_order = None
142
143             for p in range(max_p + 1):
144                 for q in range(max_q + 1):
145                     # 增加 (0,0,0) 的跳过, 因为 GARCH 会处理 const
146                     if p == 0 and q == 0:

```

```

147     continue
148     try:
149         model = ARIMA(ts, order=(p, d, q))
150         res = model.fit()
151         if res.aic < best_aic:
152             best_aic = res.aic
153             best_aic_res = res
154             best_aic_order = (p, d, q)
155         if res.bic < best_bic:
156             best_bic = res.bic
157             best_bic_res = res
158             best_bic_order = (p, d, q)
159     except Exception as e:
160         continue
161     return {
162         'best_aic_order': best_aic_order,
163         'best_bic_order': best_bic_order,
164         'best_aic_result': best_aic_res,
165         'best_bic_result': best_bic_res
166     }
167
168     # --- 残差诊断与显著性检验函数 ---
169     def arima_diagnostics(result, series_name="Series", lb_lags=[10, 20]):
170         print(f"\n--- {series_name} ARIMA 模型摘要 ---")
171         print(result.summary())
172
173         print(f"\n{series_name} 参数显著性 (p-values):")
174         pvals = result.pvalues
175         for name, pval in pvals.items():
176             print(f"   {name:20s} p = {pval:.6f} -> {'显著' if pval < 0.05 else '不显著'}")
177
178         print(f"\n{series_name} 残差 Ljung-Box 检验 (lags = {lb_lags}):")
179         for lag in lb_lags:
180             try:
181                 # 使用 statsmodels 0.13+ 的 acorr_ljungbox 格式
182                 lb = acorr_ljungbox(result.resid, lags=[lag], return_df=True)
183                 pvalue = lb['lb_pvalue'].iloc[0]
184                 print(f"   lag={lag} -> lb_stat={lb['lb_stat'].iloc[0]:.4f}, p={pvalue:.6f} -> {'通过(白噪声)' if pvalue>0.05 else '未通过(非白噪声)'}")
185             except Exception as e:
186                 print(f"   lag={lag} 检验失败: {e}")
187
188         # 残差绘图
189         resid = result.resid
190         fig, axes = plt.subplots(2, 2, figsize=(12, 8))
191         axes[0, 0].plot(resid); axes[0, 0].set_title(f"{series_name} 残差时序")
192         plot_acf(resid, lags=30, ax=axes[0, 1]); axes[0, 1].set_title(f"{series_name} 残差 ACF")
193         plot_pacf(resid, lags=30, ax=axes[1, 0]); axes[1, 0].set_title(f"{series_name} 残差 PACF")

```

```

194     sm.qqplot(resid, line='s', ax=axes[1, 1]); axes[1, 1].set_title(f"{series_name} 残差
    Q-Q 图")
195     plt.suptitle(f"{series_name} ARIMA 残差诊断")
196     plt.tight_layout(rect=[0, 0.03, 1, 0.95])
197     plt.show()
198
199     # ===== 4.1 BTC ARIMA =====
200     print("\n--- 4.1 BTC 对数收益率 ARIMA 自动寻优 (基于 AIC/BIC) ---")
201     # 确保 df_returns 存在 (在 Section 2 中创建)
202     btc_res = auto_arima_select(df_returns['btc_log_return'], d=0, max_p=3, max_q=3)
203     print(f"BTC 最优 AIC 阶数: {btc_res['best_aic_order']}")
204     print(f"BTC 最优 BIC 阶数: {btc_res['best_bic_order']}")
205
206     if btc_res['best_aic_result'] is not None:
207         arima_result = btc_res['best_aic_result'] # 用于预测
208         arima_diagnostics(arima_result, series_name="BTC 对数收益率")
209     else:
210         print("警告: 未能找到可拟合的 BTC ARIMA 模型。")
211
212     # --- 4.3 GARCH(1,1)-t 模型 (BTC 对数收益率) ---
213     print("\n--- 4.3 GARCH(1,1) 模型 (BTC 对数收益率) ---")
214     try:
215         garch_model_btc = arch_model(df_returns['btc_log_return'].dropna() * 100,
216                                     vol='Garch', p=1, o=0, q=1, dist='t')
217         garch_result_btc = garch_model_btc.fit(update_freq=5, disp='off')
218
219         print("\nBTC GARCH(1,1)-t 模型摘要:")
220         print(garch_result_btc.summary())
221
222         print("正在显示图5: BTC GARCH(1,1)-t 标准化残差...")
223         garch_std_resid_btc = garch_result_btc.resid / garch_result_btc.
            conditional_volatility
224         plt.figure(figsize=(10, 4))
225         plt.plot(garch_std_resid_btc)
226         plt.title('图5: BTC GARCH(1,1)-t 标准化残差')
227         plt.show()
228
229     except Exception as e:
230         print(f"GARCH(1,1) 拟合失败: {e}")
231
232
233     # --- 4.4 ARIMA-GARCH 联合模型 (样本内拟合与样本外预测) ---
234     print("\n--- 4.4 ARIMA-GARCH 联合模型 (样本内拟合与样本外预测) ---")
235
236     if 'arima_result' in locals() and 'garch_result_btc' in locals() and 'df_prices' in
        locals():
237         try:
238             last_price = df_prices['btc_close'].iloc[-1]
239             last_date = df_prices.index[-1]
240             forecast_horizon = 10

```



```

241
242 # 从 GARCH 结果获取 t 分布自由度
243 nu = garch_result_btc.params['nu']
244 t_critical = t.ppf(0.975, nu)
245
246 # --- 样本内拟合 ---
247 in_sample_mu = arima_result.fittedvalues
248
249 # 修正：样本内价格拟合应该基于 (t-1) 期的真实价格
250 prev_actual_prices = df_prices['btc_close'].shift(1)
251 in_sample_fitted_prices = prev_actual_prices * np.exp(in_sample_mu)
252
253 # 样本内波动率 (GARCH 条件波动率)
254 # 注意：GARCH 模型是用 100*return 拟合的，所以波动率要 /100
255 in_sample_vol = garch_result_btc.conditional_volatility / 100
256
257 # 样本内置信区间
258 in_sample_upper = prev_actual_prices * np.exp(in_sample_mu + t_critical *
in_sample_vol)
259 in_sample_lower = prev_actual_prices * np.exp(in_sample_mu - t_critical *
in_sample_vol)
260
261 # --- 样本外预测 ---
262 # 1. 均值预测 (来自 ARIMA)
263 arima_forecast = arima_result.get_forecast(steps=forecast_horizon)
264 pred_returns = arima_forecast.predicted_mean
265
266 # 2. 波动率预测 (来自 GARCH)
267 garch_forecast = garch_result_btc.forecast(horizon=forecast_horizon, reindex=False)
268 pred_volatility = np.sqrt(garch_forecast.variance.iloc[-1].values) / 100
269
270 # 3. 价格预测 (迭代)
271 cum_pred_returns = np.cumsum(pred_returns)
272 # 累计波动率 (注意：方差是可加的，波动率不是)
273 cum_pred_volatility = np.sqrt(np.cumsum(pred_volatility**2))
274
275 pred_prices = last_price * np.exp(cum_pred_returns)
276 upper_prices = last_price * np.exp(cum_pred_returns + t_critical *
cum_pred_volatility)
277 lower_prices = last_price * np.exp(cum_pred_returns - t_critical *
cum_pred_volatility)
278
279 future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1), periods=
forecast_horizon, freq='D')
280
281 price_forecast_df = pd.DataFrame({
282     'Predicted_Price': pred_prices,
283     'Lower_Price_95': lower_prices,
284     'Upper_Price_95': upper_prices
285 }, index=future_dates)

```

```

286
287     print(f"\n未来 {forecast_horizon} 天预测结果:")
288     print(price_forecast_df)
289
290     # --- 绘图 1: 全历史 ---
291     print(f"正在显示图5.4: 全历史样本内拟合与样本外预测...")
292     plt.figure(figsize=(14, 7))
293     plt.plot(df_prices.index, df_prices['btc_close'], color='blue', alpha=0.5, label='历史真实价格')
294     plt.plot(in_sample_fitted_prices.index, in_sample_fitted_prices, color='green',
295             linestyle='--', alpha=0.8, linewidth=1, label='样本内拟合价格')
296     plt.plot(future_dates, price_forecast_df['Predicted_Price'], color='red', marker='o',
297             markersize=4, label='未来预测价格')
298     plt.fill_between(future_dates, price_forecast_df['Lower_Price_95'], price_forecast_df
299                     ['Upper_Price_95'], color='red', alpha=0.15, label='未来 95% 置信区间')
300     plt.title(f'图5.4: BTC 价格全历史样本内拟合与未来 {forecast_horizon} 天预测')
301     plt.xlabel('日期')
302     plt.ylabel('BTC 价格 (USD)')
303     plt.legend(loc='upper left')
304     plt.grid(True, alpha=0.3)
305     plt.tight_layout()
306     plt.show()
307
308     # --- 绘图 2: 局部放大 (2025-09-01 至今) ---
309     print(f"正在显示图5.5: 局部样本内拟合与样本外预测 (2025年9月至今)...")
310     start_date_zoom = '2025-09-01'
311     df_zoom = df_prices[start_date_zoom:]
312     fitted_zoom = in_sample_fitted_prices[start_date_zoom:]
313     lower_zoom = in_sample_lower[start_date_zoom:]
314     upper_zoom = in_sample_upper[start_date_zoom:]
315
316     if not df_zoom.empty:
317         plt.figure(figsize=(14, 7))
318         # 历史
319         plt.plot(df_zoom.index, df_zoom['btc_close'], color='blue', marker='.', linestyle='-',
320                 linewidth=2, alpha=0.6, label='真实价格')
321         plt.plot(fitted_zoom.index, fitted_zoom, color='green', linestyle='--', linewidth
322                 =2.5, alpha=0.9, label='拟合价格')
323         plt.fill_between(fitted_zoom.index, lower_zoom, upper_zoom, color='green', alpha=0.1,
324                         label='历史 95% 置信区间')
325
326         # 预测
327         plt.plot(future_dates, price_forecast_df['Predicted_Price'], color='red', marker='o',
328                 markersize=6, linewidth=2, label='未来预测')
329         plt.fill_between(future_dates, price_forecast_df['Lower_Price_95'], price_forecast_df
330                         ['Upper_Price_95'], color='red', alpha=0.25, label='未来 95% 置信区间')
331
332         plt.title(f'图5.5: 局部样本内拟合 (带历史置信区间) 与未来预测')
333         plt.xlabel('日期')
334         plt.ylabel('BTC 价格 (USD)')

```

```

327     plt.legend(loc='upper left')
328     plt.grid(True, alpha=0.3)
329     plt.tight_layout()
330     plt.xticks(rotation=45)
331     plt.show()
332     else:
333     print(f"警告：数据中没有 {start_date_zoom} 之后的数据。")
334
335     except Exception as e:
336     print(f"模型拟合与预测失败：{e}")
337     else:
338     print(f"错误：缺少必要模型或数据。")
339
340
341     # --- 5. 多元时间序列分析 (VECM/VAR) ---
342     print("\n--- 5. 多元时间序列分析 (VECM / VAR) ---")
343     if 'df_prices' in locals() and 'df_returns' in locals():
344
345     # --- 5.1 确定最优滞后阶数 ---
346     print("\n--- 5.1 确定最优滞后阶数 (Lag Selection) ---")
347
348     # A. VAR(df_returns) 滞后阶数
349     model_returns = VAR(df_returns)
350     lag_selection_returns = model_returns.select_order(maxlags=10)
351     optimal_lag_returns = lag_selection_returns.aic
352     print(f"VAR(df_returns) 最优滞后 (AIC): {optimal_lag_returns}")
353
354     # B. VAR(df_prices) 滞后阶数
355     model_prices = VAR(df_prices)
356     lag_selection_prices = model_prices.select_order(maxlags=10)
357     optimal_lag_prices = lag_selection_prices.aic
358     print(f"VAR(df_prices) 最优滞后 (AIC): {optimal_lag_prices}")
359
360
361     # --- 5.2 协整检验 (Engle-Granger 两步法) ---
362     print("\n--- 5.2 协整检验 (Engle-Granger 两步法) ---")
363     print("H0: 两个序列没有协整关系")
364
365     # EG 两步法 (statsmodels.tsa.stattools.coint)
366     # 检验 1: btc_close = f(eth_close)
367     print("\n检验 1 (BTC ~ ETH):")
368     # trend='c' 代表包含常数项 (截距)
369     # 返回值: (t-statistic, p-value, critical_values)
370     score, pvalue, _ = sm.tsa.stattools.coint(df_prices['btc_close'], df_prices['
eth_close'], trend='c')
371
372     print(f'   t-statistic: {score:.4f}')
373     print(f'   p-value: {pvalue:.4f}')
374     if pvalue < 0.05:
375     print(f'   结论：拒绝 H0 (p < 0.05)。序列存在协整关系。')

```

```

376     else:
377         print(f'    结论: 无法拒绝 H0 (p > 0.05)。序列无协整关系。')
378
379         print("----")
380
381         # 检验 2: eth_close = f(btc_close)
382         print("检验 2 (ETH ~ BTC):")
383         score_rev, pvalue_rev, _ = sm.tsa.stattools.coint(df_prices['eth_close'], df_prices['
btc_close'], trend='c')
384
385         print(f'    t-statistic: {score_rev:.4f}')
386         print(f'    p-value: {pvalue_rev:.4f}')
387         if pvalue_rev < 0.05:
388             print(f'    结论: 拒绝 H0 (p < 0.05)。序列存在协整关系。')
389         else:
390             print(f'    结论: 无法拒绝 H0 (p > 0.05)。序列无协整关系。')
391
392
393         # --- 5.3 格兰杰因果检验 (Granger Causality Test) ---
394         print("\n--- 5.3 格兰杰因果检验 (Granger Causality Test) ---")
395         print(f"检验 'df_returns' (maxlag = {optimal_lag_returns})")
396
397         # 检验 1: ETH 是否是 BTC 的格兰杰原因
398         gc_result_eth_to_btc = grangercausalitytests(
399             df_returns[['btc_log_return', 'eth_log_return']],
400             maxlag=optimal_lag_returns,
401             verbose=False
402         )
403         f_test_p_value_1 = gc_result_eth_to_btc[optimal_lag_returns][0]['ssr_ftest'][1]
404         print(f"\nH0: ETH -/-> BTC (p-value): {f_test_p_value_1:.6f}")
405
406         # 检验 2: BTC 是否是 ETH 的格兰杰原因
407         gc_result_btc_to_eth = grangercausalitytests(
408             df_returns[['eth_log_return', 'btc_log_return']],
409             maxlag=optimal_lag_returns,
410             verbose=False
411         )
412         f_test_p_value_2 = gc_result_btc_to_eth[optimal_lag_returns][0]['ssr_ftest'][1]
413         print(f"\nH0: BTC -/-> ETH (p-value): {f_test_p_value_2:.6f}")
414
415         # --- 5.4 拟合 VAR 模型 ---
416         print("\n--- 5.4 拟合 VAR 模型 (在收益率上) ---")
417
418         if 'df_returns' in locals() and 'optimal_lag_returns' in locals():
419             try:
420                 # 1. 创建 VAR 模型实例
421                 model_var = VAR(df_returns)
422
423                 # 2. 拟合模型, 使用之前确定的最优滞后阶数 (10)
424                 print(f"正在拟合 VAR({optimal_lag_returns}) 模型...")

```

```
425     var_result = model_var.fit(optimal_lag_returns)
426
427     # 3. 输出模型摘要
428     print("\nVAR 模型摘要:")
429     print(var_result.summary())
430
431     # 4. 检查模型的稳定性 (根是否在单位圆内)
432     print("\nVAR 模型稳定性检查 (所有根的模应 < 1):")
433     is_stable = var_result.is_stable()
434     print(f"模型是否稳定: {is_stable}")
435
436     except Exception as e:
437         print(f"VAR 模型拟合失败: {e}")
438     else:
439         print("错误: 缺少必要变量, 无法拟合 VAR 模型。")
440
441
442     # --- 5.5 脉冲响应分析 (IRF) ---
443     print("\n--- 5.5 脉冲响应分析 (Impulse Response Analysis) ---")
444
445     if 'var_result' in locals():
446         try:
447             # 设定分析的期数 (例如未来 20 天)
448             irf_periods = 20
449
450             # 'orth=True' 表示我们想要 Cholesky 分解后的正交化脉冲响应
451             irf = var_result.irf(irf_periods)
452
453             print(f"正在显示图6: 正交化脉冲响应函数 (未来 {irf_periods} 期)...")
454
455             # 绘制所有组合
456             irf.plot(orth=True, signif=0.05, figsize=(12, 10))
457             plt.suptitle('图6: VAR(10) 正交化脉冲响应分析 (95% 置信区间)')
458             plt.tight_layout(rect=[0, 0.03, 1, 0.95])
459             plt.show()
460
461         except Exception as e:
462             print(f"脉冲响应分析失败: {e}")
463         else:
464             print("错误: 缺少 VAR 模型结果 (var_result), 无法进行 IRF 分析。")
465
466     # --- 5.6 方差分解分析 (FEVD) ---
467     print("\n--- 5.6 方差分解分析 (Variance Decomposition) ---")
468
469     if 'var_result' in locals():
470         try:
471             # 设定分析期数 (例如未来 10 天)
472             fevd_periods = 10
473
474             # 计算方差分解
```

```

475     fevd = var_result.fevd(fevd_periods)
476
477     print(f"\n方差分解结果 (未来 {fevd_periods} 天):")
478     print(fevd.summary())
479
480     print(f"正在显示图7: 方差分解堆叠图...")
481     # 绘制方差分解图
482     fevd.plot(figsize=(12, 8))
483     plt.suptitle(f'图7: VAR(10) 方差分解 (未来 {fevd_periods} 天)')
484     plt.tight_layout(rect=[0, 0.03, 1, 0.95])
485     plt.show()
486
487     except Exception as e:
488         print(f"方差分解分析失败: {e}")
489     else:
490         print("错误: 缺少 VAR 模型结果 (var_result), 无法进行 FEVD 分析。")
491
492     else:
493         print("错误: 缺少 'df_prices' 或 'df_returns' DataFrame, 跳过多维分析。")
494
495
496     # --- 6. 风险价值 (VaR) 计算与回测 ---
497     print("\n--- 6. 风险价值 (VaR) 计算与回测 (基于 GARCH-t 模型) ---")
498
499     if 'garch_result_btc' in locals():
500         try:
501             # 1. 获取模型参数
502             garch_params = garch_result_btc.params
503             mu = garch_params['mu']
504             nu = garch_params['nu'] # t分布自由度
505
506             # 2. 获取条件波动率 (Conditional Volatility)
507             conditional_volatility = garch_result_btc.conditional_volatility
508
509             # 3. 计算 95% VaR (alpha = 0.05)
510             alpha = 0.05
511             # t.ppf 是 t 分布的百分位点函数 (逆累积分布函数)
512             q_t = t.ppf(alpha, nu)
513
514             # VaR = E[R] + sigma_t * F_inv(alpha)
515             # 注意: GARCH 模型是用 100*R 拟合的
516             # 所以 mu 和 vol 都是基于 100*R 的尺度
517             # VaR (in 100*R scale) = mu + conditional_volatility * q_t
518             # VaR (in R scale) = (mu + conditional_volatility * q_t) / 100
519             var_95 = (mu + conditional_volatility * q_t) / 100
520
521             # 4. 创建用于绘图的 DataFrame
522             var_df = pd.DataFrame({
523                 'Actual_Return': df_returns['btc_log_return'],
524                 'VaR_95': var_95

```

```

525     }, index=df_returns.index).dropna()
526
527     # 5. 绘制 VaR 回测图
528     print("正在显示图8: BTC 每日收益率与 95% VaR 对比图...")
529     plt.figure(figsize=(12, 6))
530     plt.plot(var_df.index, var_df['Actual_Return'], color='blue', alpha=0.5, label='实际
收益率')
531     plt.plot(var_df.index, var_df['VaR_95'], color='red', linewidth=2, label='95% VaR (
GARCH-t)')
532     plt.title('图8: BTC 每日收益率 vs. 95% 动态风险价值 (VaR)')
533     plt.legend()
534     plt.grid(True, alpha=0.3)
535     plt.show()
536
537     # 6. 计算 VaR 突破次数 (Backtesting)
538     exceptions = var_df[var_df['Actual_Return'] < var_df['VaR_95']]
539     num_exceptions = len(exceptions)
540     total_obs = len(var_df)
541     actual_alpha = num_exceptions / total_obs
542
543     print(f"\nVaR 回测结果 (95% 置信水平, 理论突破率应接近 5%):")
544     print(f"    总观测天数: {total_obs}")
545     print(f"    实际突破次数: {num_exceptions}")
546     print(f"    实际突破率: {actual_alpha:.2%}")
547
548     if abs(actual_alpha - 0.05) < 0.01: # 简单判断, 实际应做 Kupiec 检验
549         print("    -> 模型表现良好 (实际突破率接近理论值 5%)")
550     else:
551         print("    -> 模型可能需要改进 (实际突破率偏离理论值)")
552
553     # 7. 预测下一交易日的 VaR
554     forecast = garch_result_btc.forecast(horizon=1)
555     next_vol = np.sqrt(forecast.variance.iloc[-1, 0])
556     next_var_95 = (mu + next_vol * q_t) / 100
557
558     print(f"\n未来一日风险预测:")
559     print(f"    下一交易日 95% VaR: {next_var_95:.4%}")
560     print(f"    (解读: 有 95% 的把握, 明天 BTC 的损失不会超过 {abs(next_var_95):.2%})")
561
562     except Exception as e:
563         print(f"VaR 计算失败: {e}")
564     else:
565         print("错误: 缺少 GARCH 模型结果 (garch_result_btc), 无法计算 VaR。")
566

```

Listing 1: Python 完整分析代码